

```

0000 separate (IDL_SEM_PHASE) if -- package body AGGREGO is use EXP_TYPE, EXPRESO use VIT_UTIL
; use DEF_UTIL; use REQ_UTIL; type ASSOC_CURSOR_TYPE is record; ASSOC_LIST : SEQ_TYPE; ASSOC : TREE; EXP : TREE;
; CHOICE_LIST : SEQ_TYPE; CHOICE : TREE; COUNT : Natural; FIRST_COUNT : Position; and record; procedure INIT_ASSOC_CURSOR (ASSOC_CURSOR : out ASSOC_CURSOR_TYPE; ASSOC_LIST : SEQ_TYPE); procedure ADVANCE_ASSOC_CURSOR (ASSOC_CURSOR : in out ASSOC_CURSOR_TYPE); function VALU
E_IS_IN_CHOICE_S (VALUE : TREE; CHOICE_S : TREE) return Boolean; procedure RESOLVE_RECORD_AGGREGATE (EXP : TREE; TYPE_STRUCT : TREE); procedure RE
SOLVE_ERRONEOUS_AGGREGATE (EXP : TREE); procedure RESOLVE_ARRAY_SUBAGGREGATE (EXP : TREE; COMP_TYPE : TREE; INDEX_LIST : SEQ_TYPE; SCALAR_LIST : in
out SEQ_TYPE; NAMED_OTHERS_OK : Boolean := False); procedure RESOLVE_STRING_SUBAGGREGATE (EXP : TREE; COMP_TYPE : TREE; INDEX : TREE; SCALAR_LIST :
in out SEQ_TYPE); procedure MAKE_NORMALIZED_LIST (AGGREGATE_ARRAY : in out AGGREGATE_ARRAY_TYPE; NORMALIZED_LIST : out SEQ_TYPE);
0010 SM_OBJ_TYPE, ID); begin if RESULT_TY in DN_PRIVATE then RESULT := D (SM_TYPE_SPEC, RESULT); else if RESULT_TY = DN_INCOM
PLETE and then D (XD_FULL_TYPE_SPEC, RESULT) /= TREE_VOID then RESULT := D (XD_FULL_TYPE_SPEC, RESULT); end if; -- procedure INIT_ASSOC_
CURSOR (ASSOC_CURSOR : out ASSOC_CURSOR_TYPE; ASSOC_LIST : SEQ_TYPE) is -- INITIALIZE-CUMULATIVE FIELDS OF ASSOC_CURSOR RECORD begin ASSO
C_CURSOR.ASSOC_LIST := ASSOC_LIST; ASSOC_CURSOR.CHOICE_LIST := (TREE_NIL, TREE_NIL); ASSOC_CURSOR.COUNT := 0; end INIT_ASSOC_CURSOR;
; procedure ADVANCE_ASSOC_CURSOR (ASSOC_CURSOR : in out ASSOC_CURSOR_TYPE) is -- ADVANCE ASSOC_CURSOR TO NEXT CHOICE begin if -- IF THE
RE ARE REMAINING CHOICES-IN CURRENT CHOICE LIST if not IS_EMPTY (ASSOC_CURSOR.CHOICE_LIST) then -- STEP TO THE NEXT ONE POP (ASSOC_CU
RSOR.CHOICE_LIST, ASSOC_CURSOR.CHOICE); ASSOC_CURSOR.COUNT := ASSOC_CURSOR.COUNT + 1; -- ELSE IF THERE ARE REMAINING ASSOCIATIONS els
e if not IS_EMPTY (ASSOC_CURSOR.ASSOC_LIST) then -- STEP TO THE NEXT ASSOCIATION POP (ASSOC_CURSOR.ASSOC_LIST, ASSOC_CURSOR.ASSOC);
0020 ASSOC_CURSOR.COUNT := ASSOC_CURSOR.COUNT + 1; ASSOC_CURSOR.FIRST_COUNT := ASSOC_CURSOR.COUNT; -- IF IT IS A NAMED ASSOCIATION if AS
SOC_CURSOR.ASSOC_TY = DN_NAMED then -- SAVE-THE EXPRESSION ASSOC_CURSOR.EXP := D (AS_EXP, ASSOC_CURSOR.ASSOC); -- GET THE LIST OF
CHOICES ASSOC_CURSOR.CHOICE_LIST := LIST (D (AS_CHOICE_S, ASSOC_CURSOR.ASSOC)); -- STEP-TO THE FIRST CHOICE POP (ASSOC_CURSOR.
CHOICE_LIST, ASSOC_CURSOR.CHOICE); -- ELSE-- SINCE IT IS NOT A NAMED ASSOCIATION else -- SAVE-THE EXPRESSION ASSOC_CURSOR.EXP
:= ASSOC_CURSOR.ASSOC; SET CHOICE TO VOID; ASSOC_CURSOR.CHOICE := TREE_VOID; -- ELSE-- SINCE THERE ARE NO MORE ASSOCIATIONS
end if; else -- SET THE ASSOC FIELD TO VOID TO INDICATE TERMINATION ASSOC_CURSOR.ASSOC := TREE_VOID; end if; end ADVANCE_
SSOC_CURSOR;
; function COUNT_AGGREGATE_CHOICES (ASSOC_S : TREE) return Natural is -- COUNT THE NUMBER OF DISTINCT CHOICES IN A LIST OF ASSOCIATIONS
; -- (EITHER IN A DISCR
MINANT CONSTRAINT-OR AN AGGREGATE) ASSOC_CURSOR : ASSOC_CURSOR_TYPE; begin -- STEP THROUGH CHOICES INIT_ASSOC_CURSOR (ASSOC_CURSOR,
LIST (ASSOC_S)); loop ADVANCE_ASSOC_CURSOR (ASSOC_CURSOR); exit when ASSOC_CURSOR.ASSOC = TREE_VOID; end loop; -- RETURN
THE-COUNT FROM THE CURSOR return ASSOC_CURSOR.COUNT; end COUNT_AGGREGATE_CHOICES;
0030 -- procedure SPREAD_ASSOC_S (ASSOC_S : TREE; AGGREGATE_ARRAY : in out AGGREGATE_ARRAY_TYPE) is
-- AN AGGREGATE_ARRAY (WHICH IS KNOWN TO BE OF CORRECT SIZE) ASSOC_CURSOR : AS
SOC_CURSOR_TYPE; begin -- FOR EACH ARRAY ELEMENT AND CORRESPONDING-CHOICE INIT_ASSOC_CURSOR (ASSOC_CURSOR, LIST (ASSOC_S)); for I in
AGGREGATE_ARRAY_RANGE loop ADVANCE_ASSOC_CURSOR (ASSOC_CURSOR); -- FILL IN FIELDS OF AGGREGATE_ARRAY AGGREGATE_ARRAY (I).FIRST
:= ASSOC_CURSOR.FIRST_COUNT; AGGREGATE_ARRAY (I).CHOICE := ASSOC_CURSOR.CHOICE; AGGREGATE_ARRAY (I).SEEN := False; AGGREGATE_A
RRAY (I).RESOLVED := False; AGGREGATE_ARRAY (I).ID := TREE_VOID; -- FILL IN EXP AND EVALUATE TYPES FOR FIRST CHOICE OF ASSOC
I f I = ASSOC_CURSOR.FIRST_COUNT then AGGREGATE_ARRAY (I).ASSOC := ASSOC_CURSOR.ASSOC; AGGREGATE_ARRAY (I).EXP := ASSOC_CURSOR.EXP;
EVAL_EXP_TYPES (AGGREGATE_ARRAY (I).EXP, AGGREGATE_ARRAY (I).TYPESET); end if; end loop; end SPREAD_ASSOC_S;
; procedure WALK_RECORD_DECL_S (EXP : TREE; DECL_S : TREE; AG
GREGATE_ARRAY : in out AGGREGATE_ARRAY_TYPE; NORMALIZED_LIST : in out SEQ_TYPE; LAST_POSITIONAL : in out Natural) is -- WALK ONE SEQUENCE OF CO
MPONENT-DECLARATIONS FOR A RECORD -- (THERE IS ONE SUCH FOR DISCRIMINANTS-AND ONE FOR COMP_LIST) PARAM_CURSOR : PARAM_CURSOR_TYPE; NAM
E_SUB : Natural; CHOICE : TREE; begin -- FOR EACH COMPONENT DECLARED IN-THE DECL_S INIT_PARAM_CURSOR (PARAM_CURSOR, LIST (
DECL_S)); loop ADVANCE_PARAM_CURSOR (PARAM_CURSOR); exit when PARAM_CURSOR.ID = TREE_VOID; -- IF-IT MATCHES A POSITIONAL PARAM
ETER if LAST_POSITIONAL < AGGREGATE_ARRAY.LAST and then AGGREGATE_ARRAY (LAST_POSITIONAL + 1).CHOICE = TREE_VOID then -- MARK-POSITIONAL PA
RAMETER SEEN LAST_POSITIONAL := LAST_POSITIONAL + 1; AGGREGATE_ARRAY (LAST_POSITIONAL).SEEN := True; AGGREGATE_ARRAY (LAST
_POSITIONAL).RESOLVED := True; AGGREGATE_ARRAY (LAST_POSITIONAL).ID := PARAM_CURSOR.ID; -- CHECK TYPE AND-RESOLVE EXPRESSION RE
QUIRE_TYPE (GET_BASE_TYPE (PARAM_CURSOR.ID), AGGREGATE_ARRAY (LAST_POSITIONAL).EXP, AGGREGATE_ARRAY (LAST_POSITIONAL).TYPESET);
0040 Y (LAST_POSITIONAL).EXP := RESOLVE_EXP_OR_AGGREGATE (AGGREGATE_ARRAY (LAST_POSITIONAL).EXP, GET_SUBTYPE_OF_ID (PARAM_CURSOR.ID), NAMED_OTHERS_OK => Tr
ue); -- ADD EXPRESSION-TO NORMALIZED LIST NORMALIZED_LIST := APPEND (NORMALIZED_LIST, AGGREGATE_ARRAY (LAST_POSITIONAL).EXP); -- E
LSE-- SINCE NO MORE POSITIONAL PARAMETERS else -- SEARCH FOR MATCHING NAME NAMED_SUB := LAST_POSITIONAL; loop NAMED_SUB + 1; -- exit when NAMED_SUB => AGGREGATE_ARRAY.LAST; -- CHOICE := AGGREGATE_ARRAY (NAMED_SUB).CHOICE; -- exit when CHOICE.TY = DN_CHOICE_OTHERS;
-- exit when not AGGREGATE_ARRAY (NAMED_SUB).SEEN and then CHOICE.TY = DN_CHOICE_EXP and then D (AS_EXP, CHOICE).TY in CLASS_DESIGNATOR and then D (LX_
SYMREP, D (AS_EXP, CHOICE)) = D (LX_SYMREP, PARAM_CURSOR.ID); end loop; -- IF MATCH WAS FOUND if NAMED_SUB < AGGREGATE_ARRAY.LA
ST then -- MARK NAMED PARAMETER SEEN AGGREGATE_ARRAY (NAMED_SUB).SEEN := True; AGGREGATE_ARRAY (NAMED_SUB).ID := PARAM_CURSOR.ID; -- REP
LACE CHOICE-EXP EXPRESSION WITH USED_NAME_ID if AGGREGATE_ARRAY (NAMED_SUB).CHOICE.TY = DN_CHOICE_EXP then D (AS_EXP, AGGREGATE_ARRAY (NAMED_SUB).
CHOICE, MAKE_USED_NAME_ID FROM OBJECT D (AS_EXP, AGGREGATE_ARRAY (NAMED_SUB).CHOICE)); D (SM_DEFN, D (AS_EXP, AGGREGATE_ARRAY (NAMED_SUB).CHOICE),
PARAM_CURSOR.ID); end if; -- CHECK TYPE (FOR FIRST CHOICE OF AN ASSOCIATION) -- (NOTE. GIVES-ERROR IF CONFLICTING TYPES IN-ASSOC) -- NAM
E_SUB := AGGREGATE_ARRAY (NAMED_SUB).FIRST; -- REQUIRE_TYPE (GET_BASE_TYPE (PARAM_CURSOR.ID), AGGREGATE_ARRAY (NAMED_SUB).EXP, AGGREGATE_ARRAY (NAMED_S
UB).TYPESET); -- RESOLVE, IF THIS-EXP NOT ALREADY RESOLVED if not AGGREGATE_ARRAY (NAMED_SUB).RESOLVED then AGGREGATE_ARRAY (NAMED_SUB).EXP
:= RESOLVE_EXP_OR_AGGREGATE (AGGREGATE_ARRAY (NAMED_SUB).EXP, GET_SUBTYPE_OF_ID (PARAM_CURSOR.ID), NAMED_OTHERS_OK => True); AGGREGATE_ARRAY (NAMED_S
UB).RESOLVED := True; end if; -- ADD EXPRESSION TO NORMALIZED LIST NORMALIZED_LIST := APPEND (NORMALIZED_LIST, AGGREGATE_ARRAY (NAMED_SUB
).EXP); -- ELSE -- SINCE NO-MATCH WAS-FOUND else -- INDICATE ERROR-ERROR (D (LX_SRCPOS, EXP), "NO VALUE FOR COMPONENT - " & PRINT_N
AME (D (LX_SYMREP, PARAM_CURSOR.ID))); end if; end loop; end WALK_RECORD_DECL_S;
; procedure RESOLVE_RECORD_ASSOC_S (ASSOC_S : TREE; AGGREGATE_ARRAY : in out AGG
REGATE_ARRAY_TYPE) is -- RESOLVE ELEMENTS OF AN ASSOC_S-FOR AN AGGREGATE -- (INDIVIDUAL EXPRESSIONS HAVE BEEN RESOLVED) NEW_ASSO
C : TREE; NEW_ASSOC_LIST : SEQ_TYPE := (TREE_NIL, TREE_NIL); begin -- FOR EACH ARRAY ELEMENT AND CORRESPONDING-CHOICE for I in
AGGREGATE_ARRAY_RANGE loop -- IF-ELEMENT IS FIRST CHOICE OF AN-ASSOCIATION if I = AGGREGATE_ARRAY (I).FIRST then -- MAKE-SURE THAT
-EXPRESSION HAS BEEN-RESOLVED if not AGGREGATE_ARRAY (I).RESOLVED then AGGREGATE_ARRAY (I).EXP := RESOLVE_EXP (AGGREGATE_ARRAY (I).EXP, TREE_
VOID); -- REPLACE RESOLVED-EXPRESSION end if; if AGGREGATE_ARRAY (I).CHOICE = TREE_VOID then NEW_ASSOC := AGGREGATE_ARRAY (I).EX
P; else NEW_ASSOC := AGGREGATE_ARRAY (I).ASSOC; D (AS_EXP, NEW_ASSOC, AGGREGATE_ARRAY (I).EXP); end if; -- ADD ASSOCIATION TO
NEW LIST; NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, NEW_ASSOC); end if; -- CHECK THAT CHOICE EXISTED IN TYPE if not AGGREGA
TE_ARRAY (I).SEEN then if AGGREGATE_ARRAY (I).CHOICE = TREE_VOID then ERROR (D (LX_SRCPOS, NEW_ASSOC), "NO MATCHING COMPONENT"); else f
AGGREGATE_ARRAY (I).CHOICE.TY = DN_CHOICE_EXP then if D (AS_EXP, AGGREGATE_ARRAY (I).CHOICE).TY = DN_EXP_OBJECT_ID then ERROR (D (LX_SRCPOS, AG
GREGATE_ARRAY (I).CHOICE), "NO MATCHING COMPONENT FOR - " & PRINT_NAME (D (LX_SYMREP, D (AS_EXP, AGGREGATE_ARRAY (I).CHOICE))); else ERROR (D (LX_
SRCPOS, AGGREGATE_ARRAY (I).CHOICE), "SIMPLE NAME REQUIRED"); end if; else if AGGREGATE_ARRAY (I).CHOICE.TY = DN_CHOICE_EXP then if D (LX_
SRCPOS, AGGREGATE_ARRAY (I).CHOICE), "RANGE NOT ALLOWED"; else -- SINCE KIND (...) = DN_CHOICE_OTHERS then ERROR (D (LX_SRCPOS, AGGREGATE_ARRAY (I
).CHOICE), "NO MATCHING COMPONENT FOR OTHERS"); end if; end if; end loop; -- INSERT RESOLVED LIST IN ASSOC_S LIST (ASSO
C_S, NEW_ASSOC_LIST); end RESOLVE_RECORD_ASSOC_S;
; function RESOLVE_EXP_OR_AGGREGATE (EXP : TREE; SUBTYPE_SPEC : TREE; NAMED_OTHERS_OK : Boolean) return TREE is SCALAR_LIST : SEQ_TYP
E := (TREE_NIL, TREE_NIL); begin if SUBTYPE_SPEC.TY = DN_CONSTRAINED_ARRAY then -- $$$ NEED TO-PASS SUBTYPES OF INDEXES if EXP.TY =
DN_AGGREGATE then RESOLVE_ARRAY_SUBAGGREGATE (EXP, D (SM_COMP_TYPE, GET_BASE_STRUCT (SUBTYPE_SPEC)), LIST (D (SM_INDEX_S, GET_BASE_STRUCT (SU
BTYPE_SPEC))), SCALAR_LIST, NAMED_OTHERS_OK); D (SM_EXP_TYPE, EXP, SUBTYPE_SPEC); else if EXP.TY = DN_STRING_LITERAL then RESOLVE_
STRING_SUBAGGREGATE (EXP, D (SM_COMP_TYPE, GET_BASE_STRUCT (SUBTYPE_SPEC)), HEAD (LIST (D (SM_INDEX_S, GET_BASE_STRUCT (SUBTYPE_SPEC))), SCALAR_LIST
); D (SM_EXP_TYPE, EXP, SUBTYPE_SPEC); else return RESOLVE_EXP (EXP, GET_BASE_TYPE (SUBTYPE_SPEC)); end if; else
return RESOLVE_EXP (EXP, GET_BASE_TYPE (SUBTYPE_SPEC)); end if; return EXP; end RESOLVE_EXP_OR_AGGREGATE;
; procedure RESOLVE_AGGREGATE (EXP : TREE; TYPE_SPEC : TREE) is TYPE_
STRUCT : TREE := GET_BASE_STRUCT (TYPE_SPEC); SCALAR_LIST : SEQ_TYPE := (TREE_NIL, TREE_NIL); begin if TYPE_STRUCT.TY = DN_RECORD then
RESOLVE_RECORD_AGGREGATE (EXP, TYPE_STRUCT); D (SM_EXP_TYPE, EXP, TYPE_SPEC); else if TYPE_STRUCT.TY = DN_ARRAY then RESOLVE_ARRAY_
SUBAGGREGATE (EXP, D (SM_COMP_TYPE, GET_BASE_STRUCT (TYPE_SPEC)), LIST (D (SM_INDEX_S, GET_BASE_STRUCT (TYPE_SPEC))), SCALAR_LIST); D (SM_EXP_T
YPE, EXP, TYPE_SPEC); else RESOLVE_ERRONEOUS_AGGREGATE (EXP); end if; end RESOLVE_AGGREGATE;
0090 -- procedure RESOLVE_STRING (EXP : TREE; TYPE_SPEC : TREE) is SCALAR_LIST : SE
Q_TYPE := (TREE_NIL, TREE_NIL); TYPE_STRUCT : TREE := GET_BASE_TYPE (TYPE_SPEC); COMP_TYPE : TREE := TREE_VOID; INDEX_TYPE : TR
EE := TREE_VOID; begin if TYPE_STRUCT /= TREE_VOID then COMP_TYPE := D (SM_COMP_TYPE, TYPE_STRUCT); INDEX_TYPE := HEAD (LIST (D
(SM_INDEX_S, TYPE_STRUCT))); end if; RESOLVE_STRING_SUBAGGREGATE (EXP, GET_BASE_TYPE (COMP_TYPE), INDEX_TYPE, SCALAR_LIST); end RESOLVE_STR
ING;
; procedure RESOLVE_ERRONEOUS_AGG
REGATE (EXP : TREE) is -- TYPE WRONG-FOR AGGREGATE OR UNRESOLVED -- CHECK EXPRESSIONS ANYWAY GENERAL_ASSOC_S : constant tree := D (AS_
GENERAL_ASSOC_S, EXP); ASSOC_COUNT : Natural := COUNT_AGGREGATE_CHOICES (GENERAL_ASSOC_S); AGGREGATE_ARRAY : AGGREGATE_ARRAY_TYPE (1
ASSOC_COUNT); TEMP_EXP : TREE; begin D (SM_EXP_TYPE, EXP, TREE_VOID); SPREAD_ASSOC_S (GENERAL_ASSOC_S, AGGREGATE_ARRAY); for I in A
GGREGATE_ARRAY_RANGE loop if AGGREGATE_ARRAY (I).FIRST = I then TEMP_EXP := RESOLVE_EXP (AGGREGATE_ARRAY (I).EXP, TREE_VOID); end
if; end loop; end RESOLVE_ERRONEOUS_AGGREGATE;
0100 -- procedure RESOLVE_RECORD_AGGREGATE (EXP : TREE; TYPE_STRUCT : TREE) is GENERAL_ASSOC_S : constant tree := D (AS_GENERAL_ASSOC_S, EXP);
ASSOC_COUNT : Natural := COUNT_AGGREGATE_CHOICES (GENERAL_ASSOC_S); AGGREGATE_ARRAY : AGGREGATE_ARRAY_TYPE (1 ASSOC_COUNT); LAST_P
OSITIONAL : Natural := 0; COMP_LIST : TREE := D (SM_COMP_LIST, TYPE_STRUCT); VARIANT_PART : TREE; NORMALIZED_LIST : SEQ_TYPE
:= (TREE_NIL, TREE_NIL); begin D (SM_DISCRETE_RANGE, EXP, TREE_VOID); SPREAD_ASSOC_S (GENERAL_ASSOC_S, AGGREGATE_ARRAY); WALK_RECOR
D_DECL_S (EXP, D (SM_DISCRIMINANT_S, TYPE_STRUCT), AGGREGATE_ARRAY, NORMALIZED_LIST, LAST_POSITIONAL); while COMP_LIST /= TREE_VOID loop WAL
K_RECORD_DECL_S (EXP, D (AS_DECL_S, COMP_LIST), AGGREGATE_ARRAY, NORMALIZED_LIST, LAST_POSITIONAL); VARIANT_PART := D (AS_VARIANT_PART, COMP_LIST
); COMP_LIST := TREE_VOID; if VARIANT_PART /= TREE_VOID then declare DSCRMT_ID : constant tree := D (SM_DEFN, D (AS_NAME,
VARIANT_PART)); DSCRMT_EXP : TREE := TREE_VOID; DSCRMT_VALUE : TREE; VARIANT_LIST : SEQ_TYPE := LIST (D (AS_VARIANT_S, VARIANT_PART
)); VARIANT := TREE; begin for I in AGGREGATE_ARRAY_RANGE loop if (DSCRMT_ID = AGGREGATE_ARRAY (I).
```

```

e; end VALUE_IS_IN_CHOICE_S; --|
ure RESOLVE_ARRAY_SUBAGGREGATE (EXP : TREE; COMP_TYPE : TREE; INDEX_LIST : SEQ_TYPE; SCALAR_LIST : in-out SEQ_TYPE; NAMED_OTHERS_OK : Boolean := False
) is
  GENERAL_ASSOC_S : TREE := D (AS_GENERAL_ASSOC_S, EXP); INDEX := HEAD (INDEX_LIST); INDEX_TAIL := SEQ_TYPE := TAIL (INDEX_LIST);
  ASSOC_COUNT := Natural := COUNT_AGGREGATE_CHOICES (GENERAL_ASSOC_S); AGGREGATE_ARRAY := AGGREGATE_ARRAY_TYPE (1 .. ASSOC_COUNT); TYPESET
  T := TYPESET_TYPE; NEW_ASSOC_LIST := SEQ_TYPE := (TREE_NIL, TREE_NIL); CHOICE := TREE; INDEX_TYPE := TREE; POSITIONAL_SEEN := Boolean := F
alse; NAMED_SEEN := Boolean := False; OTHERS_SEEN := Boolean := False; IS_RANGE := Boolean; begin
  D (SM_DISCRETE_RANGE, EXP, TREE_VOID); -- SPREAD AGGREGATE INTO ARRAY SPREAD_ASSOC_S (GENERAL_ASSOC_S, AGGREGATE_ARRAY); -- RES
OLVE SUBEXPRESSIONS if IS_EMPTY (INDEX_TAIL) then for I in AGGREGATE_ARRAY' RANGE loop if AGGREGATE_ARRAY (I).FIRST = I then TYPESET
0130 := AGGREGATE_ARRAY (I).TYPESET; REQUIRE_TYPE (GET_BASE_TYPE (COMP_TYPE), AGGREGATE_ARRAY (I).EXP, TYPESET); AGGREGATE_ARRAY (I).EXP := RESOLVE_EXP
OR_AGGREGATE (AGGREGATE_ARRAY (I).EXP, COMP_TYPE, NAMED_OTHERS_OK = True); end if; else for I in AGGREGATE_ARRAY' RA
NGE loop if AGGREGATE_ARRAY (I).FIRST = I then if AGGREGATE_ARRAY (I).EXP.TY = DN_AGGREGATE then RESOLVE_ARRAY_SUBAGGREGATE (AGGREGATE_ARR
AY (I).EXP, COMP_TYPE, INDEX_TAIL, SCALAR_LIST, NAMED_OTHERS_OK); else if AGGREGATE_ARRAY (I).EXP.TY = DN_STRING_LITERAL and then IS_EMPTY (TAIL (INDEX
_TAIL)) and then (IS_CHARACTER_TYPE (GET_BASE_TYPE (COMP_TYPE)) or else COMP_TYPE = TREE_VOID) then RESOLVE_STRING_SUBAGGREGATE (AGGREGATE_ARRAY (I
).EXP, COMP_TYPE, HEAD (INDEX_TAIL), SCALAR_LIST); else ERROR (D (LX_SRCPOS, AGGREGATE_ARRAY (I).EXP), "INVALID FORM FOR SUBAGGREGATE"); EVAL_E
XP_TYPES (AGGREGATE_ARRAY (I).EXP, TYPESET); AGGREGATE_ARRAY (I).EXP := RESOLVE_EXP (AGGREGATE_ARRAY (I).EXP, TREE_VOID); end if; end if;
end loop; end if; -- CONSTRUCT NEW ASSOC LIST for I in AGGREGATE_ARRAY' RANGE loop if AGGREGATE_ARRAY (I).FIRST = I then
if AGGREGATE_ARRAY (I).CHOICE = TREE_VOID then AGGREGATE_ARRAY (I).ASSOC := AGGREGATE_ARRAY (I).EXP; POSITIONAL_SEEN := True; el
se D (AS_EXP, AGGREGATE_ARRAY (I).ASSOC, AGGREGATE_ARRAY (I).EXP); if AGGREGATE_ARRAY (I).CHOICE.TY = DN_CHOICE_OTHERS then OTHERS_SEEN := True; el
0140 else NAMED_SEEN := True; end if; end if; NEW_ASSOC_LIST := APPEND (NEW_ASSOC_LIST, AGGREGATE_ARRAY (I).ASSOC); end if;
end loop; -- REPLACE LIST IN GENERAL_ASSOC_S WITH RESOLVED LIST if POSITIONAL_SEEN then LIST (GENERAL_ASSOC_S, NEW_ASSOC_LIST);
end if; -- IF A NAMED ASSOCIATION WAS SEEN if NAMED_SEEN then if POSITIONAL_SEEN then ERROR (D (LX_SRCPOS, EXP), "POSITIONA
L AND NAMED ASSOCIATIONS NOT ALLOWED"); else if not NAMED_OTHERS_OK and OTHERS_SEEN then ERROR (D (LX_SRCPOS, EXP), "POSITIONA
T ALLOWED WITH OTHERS"); end if; -- EVALUATE CHOICES if INDEX.TY = DN_INDEX then -- (NOTE. ANON INDEX BASE TYPE MAY HAVE VOID
EXPRESSION) INDEX_TYPE := D (SM_TYPE_SPEC, INDEX); else INDEX_TYPE := INDEX; end if; INDEX_TYPE := GET_BASE_TYPE (I
NDEX_TYPE); for I in AGGREGATE_ARRAY' RANGE loop CHOICE := AGGREGATE_ARRAY (I).CHOICE; case CHOICE.TY is when DN_CHOICE_EXP =>
EVAL_EXP_SUBTYPE_TYPES (D (AS_EXP, CHOICE), TYPESET, IS_RANGE); REQUIRE_TYPE (INDEX_TYPE, D (AS_EXP, CHOICE), TYPESET); if IS_RANGE then
declare NEW_CHOICE : TREE := MAKE (DN_CHOICE_RANGE); begin
D (AS_DISCRETE_RANGE, NEW_CHOICE, D (AS_EXP, CH
OICE), GET_THE_TYPE (TYPESET)); D (LX_SRCPOS, NEW_CHOICE, D (LX_SRCPOS, CHOICE); CHOICE := NEW_CHOICE; end; AGGREGATE_ARRAY
0150 (I).CHOICE := CHOICE; if AGGREGATE_ARRAY (I).FIRST = I and then IS_EMPTY (TAIL (LIST (D (AS_CHOICE_S, AGGREGATE_ARRAY (I).ASSOC)))) then
-- REPLACE SINGLETON LIST LIST (D (AS_CHOICE_S, AGGREGATE_ARRAY (I).ASSOC), SINGLETON (CHOICE)); end if; else D (AS_EXP, CHOICE
, RESOLVE_EXP (D (AS_EXP, CHOICE), TYPESET)); end if; when DN_CHOICE_RANGE => EVAL_DISCRETE_RANGE (D (AS_DISCRETE_RANGE, CHOICE), TYPESET);
REQUIRE_TYPE (INDEX_TYPE, D (AS_DISCRETE_RANGE, CHOICE), TYPESET); D (AS_DISCRETE_RANGE, CHOICE, RESOLVE_DISCRETE_RANGE (D (AS_DISCRETE_RANGE, CHOI
CE), GET_THE_TYPE (TYPESET))); when DN_CHOICE_OTHERS => null; when others => null; end case; end loop; MAKE_NORMALIZED_LIS
T (AGGREGATE_ARRAY, NEW_ASSOC_LIST); GENERAL_ASSOC_S := COPY_NODE (GENERAL_ASSOC_S); LIST (GENERAL_ASSOC_S, NEW_ASSOC_LIST); end if;
D (SM_NORMALIZED_COMP_S, EXP, GENERAL_ASSOC_S); end RESOLVE_ARRAY_SUBAGGREGATE;
--|
out SEQ_TYPE) is begin
D (SM_EXP_TYPE, EXP, TREE_VOID); D (SM_DISCRETE_RANGE, EXP, TREE_VOID);
--| procedure MAKE_NORMALIZED_LIST (AGGRE
0160 GATE_ARRAY : in out AGGREGATE_ARRAY_TYPE; NORMALIZED_LIST : out SEQ_TYPE) is
-- MAKES NORMALIZED LIST FOR ARRAY AGGREGATE AGGREGATE_ITEM :=
AGGREGATE_ITEM_TYPE; NEW_NORMALIZED_LIST : SEQ_TYPE := (TREE_NIL, TREE_NIL); NON_STATIC_SEEN := Boolean := False; CHOICE := TRE
E; RANGE_NODE := TREE; begin
NORMALIZED_LIST := (TREE_NIL, TREE_NIL); -- FOR EACH CHOICE for II in AGGREGATE_ARRAY' RANGE loop
if II < AGGREGATE_ARRAY' LAST and then AGGREGATE_ARRAY (II + 1).FIRST = II then -- PREMIER MAIS PAS SEUL declare CHOICE_S :
TREE := MAKE (DN_CHOICE_S); NAMED := TREE := MAKE (DN_NAMED); begin
LIST (CHOICE_S, SINGLETON (AGGREGATE_ARRAY (II).CHOICE)); D (LX_SRCPOS
S, CHOICE_S, D (LX_SRCPOS, AGGREGATE_ARRAY (II).CHOICE)); D (AS_EXP, NAMED, D (AS_EXP, AGGREGATE_ARRAY (II).ASSOC)); D (AS_CHOICE_S, NAMED, CHOI
CE_S); D (LX_SRCPOS, NAMED, D (LX_SRCPOS, AGGREGATE_ARRAY (II).ASSOC)); AGGREGATE_ARRAY (II).ASSOC := NAMED; end;
else if PAS PRE
MIER AGGREGATE_ARRAY (II).CHOICE.TY = DN_CHOICE_RANGE and then D (AS_DISCRETE_RANGE, AGGREGATE_ARRAY (II).CHOICE).TY = DN_DISCRETE_SUBTYPE the
n AGGREGATE_ARRAY (II).ASSOC := COPY_NODE (AGGREGATE_ARRAY (II).ASSOC); -- POURRAIT CHANGER DE-TYPE DISCRET RANGE end if; else
0170 -- PAS LE PREMIER CHOICE declare CHOICE_S : TREE := MAKE (DN_CHOICE_S); NAMED := TREE := MAKE (DN_NAMED); begin
LIST (CHOICE_S, SINGLETON (AGGREGATE_ARRAY (II).CHOICE)); D (LX_SRCPOS, CHOICE_S, D (LX_SRCPOS, AGGREGATE_ARRAY (II).CHOICE)); D (AS_EXP, NAMED, D (AS_EXP, AGGR
EGATE_ARRAY (AGGREGATE_ARRAY (II).FIRST).ASSOC)); D (AS_CHOICE_S, NAMED, CHOICE_S); D (LX_SRCPOS, NAMED, D (LX_SRCPOS, AGGREGATE_ARRAY (II).CHOICE));
AGGREGATE_ARRAY (II).ASSOC := NAMED; end; end if; -- REUSE EXP AS-VALUE OF STATIC CHOICE -- COMPUTE FIRST STATIC VALUE F
OR CHOICE CHOICE := AGGREGATE_ARRAY (II).CHOICE; if CHOICE.TY = DN_CHOICE_EXP then -- (IT'S A CHOICE_EXP) AGGREGATE_ARRAY (II
).EXP := GET_STATIC_VALUE (D (AS_EXP, CHOICE)); else if CHOICE.TY = DN_CHOICE_RANGE then -- (IT'S A CHOICE_RANGE) AGGREGATE_ARRAY (II
).EXP := TREE_VOID; RANGE_NODE := D (AS_DISCRETE_RANGE, CHOICE); if RANGE_NODE.TY = DN_DISCRETE_SUBTYPE then -- (RANGE-GIVEN A
S DISCRETE SUBTYPE -- FIND RANGE) RANGE_NODE := D (AS_SUBTYPE_INDICATION, RANGE_NODE); if D (AS_CONSTRAINT, RANGE_NODE) /= TREE_VOID then RANGE_N
ODE := D (AS_CONSTRAINT, RANGE_NODE); else RANGE_NODE := D (AS_NAME, RANGE_NODE); if RANGE_NODE.TY = DN_SELECTED then RANGE_NODE := D (AS
DESIGNATOR, RANGE_NODE); end if; RANGE_NODE := D (SM_DEFN, RANGE_NODE); if RANGE_NODE.TY in CLASS_TYPE_NAME then RANGE_NODE := D (SM_TYP
E_SPEC, RANGE_NODE); end if; if RANGE_NODE.TY in CLASS_SCALAR then RANGE_NODE := D (SM_RANGE, RANGE_NODE); end if; end if; if RANGE_N
ODE.TY = DN_RANGE and then GET_STATIC_VALUE (D (AS_EXP1, RANGE_NODE)) /= TREE_VOID and then GET_STATIC_VALUE (D (AS_EXP2, RANGE_NODE)) /= TREE_VOID th
en CHOICE := COPY_NODE (CHOICE); -- LE SOUS TYPE DISCRET EST-STATIQUE, REMPLACER-PAR UNE RANGE declare CHOICE_S : TREE := MAKE (DN_CHOIC
E_S); begin
LIST (CHOICE_S, SINGLETON (CHOICE)); D (LX_SRCPOS, CHOICE_S, TREE_VOID); D (AS_CHOICE_S, AGGREGATE_ARRAY (II).ASSOC, CHO
ICE_S); end; D (AS_DISCRETE_RANGE, CHOICE, RANGE_NODE); end if; end if; -- GET STATIC VALUE FOR FIRST ELEMENT OF RANGE if
RANGE_NODE.TY = DN_RANGE then AGGREGATE_ARRAY (II).EXP := GET_STATIC_VALUE (D (AS_EXP1, RANGE_NODE)); else AGGREGATE_ARRAY (II).EXP := TREE
VOID; end if; else if CHOICE.TY = DN_CHOICE_OTHERS then AGGREGATE_ARRAY (II).EXP := TREE_VOID; else -- (NOT-CHOICE_ANYT
HING; ERROR MUST HAVE BEEN REPORTED) return; end if; -- CHECK FOR ILLEGAL NON-STATIC if AGGREGATE_ARRAY (II).EXP = TREE_VOI
D and CHOICE.TY = DN_CHOICE_OTHERS and AGGREGATE_ARRAY' LENGTH > 1 then NON_STATIC_SEEN := True; ERROR (D (LX_SRCPOS, CHOICE), "CHOICE
MUST BE STATIC"); end if; end loop; if NON_STATIC_SEEN then return; end if; -- SORT THE ENTRIES for II in AGGRE
0190 GATE_ARRAY' FIRST + 1 .. AGGREGATE_ARRAY' LAST loop exit when AGGREGATE_ARRAY (JJ).EXP = TREE_VOID; -- OTHERS for JJ in reverse AGGREG
ATE_ARRAY' FIRST .. II - 1 loop exit when UARITH "<" (AGGREGATE_ARRAY (JJ).EXP, AGGREGATE_ARRAY (JJ + 1).EXP); AGGREGATE_ITEM := AG
GREGATE_ARRAY (JJ); AGGREGATE_ARRAY (JJ) := AGGREGATE_ARRAY (JJ + 1); AGGREGATE_ITEM := AGGREGATE_ITEM; end loop;
end loop; -- CONSTRUCT THE NEW LIST for II in AGGREGATE_ARRAY' RANGE loop NEW_NORMALIZED_LIST := APPEND (NEW_NORMALIZED_LIST, AGGR
EGATE_ARRAY (II).ASSOC); end loop; NORMALIZED_LIST := NEW_NORMALIZED_LIST; end MAKE_NORMALIZED_LIST;
end AGGREGATE_ARRAY;

```